

NutriDesi — System Architecture Explainer

For: Non-technical stakeholders evaluating the project for funding/partnership **Date:** July 2026 **Status:** Live beta with 4 founding members, 65/65 eval suite pass rate

What NutriDesi Does (One Paragraph)

NutriDesi is a calorie and protein tracker that lives entirely inside WhatsApp. Users text what they ate — in casual Hindi-English ("2 roti aur dal") — and get back a calorie breakdown in under 2 seconds. No app to download, no account to create, no form to fill. It works because WhatsApp is already open on every Indian phone, every day. The hypothesis: people fail at calorie tracking not because databases are bad, but because opening a separate app at 9pm after dinner is friction they won't do.

User Acquisition Funnel — Instagram to Active User

The Channel

NutriDesi acquires users through the founder's personal Instagram (stories + reels). No paid marketing. The content is trial reels showing real bot conversations — "watch me track my entire day's food in 30 seconds on WhatsApp."

Funnel Stages with Live Metrics

``

000		
STAGE	METRIC	CONVERSION
1. Reel/Story impressions	[from IG]	—
↓		
2. DM automation triggered	[from Meta]	view → DM rate
(viewer DMs "hi" to page)		
↓		
3. Joined sandbox OR waitlist	94 users	DM → join rate
(sent join code / signed up)	+ 4 waitlist	
↓		
4. Logged at least 1 food	87 users	92% of joiners
↓		
5. Returned Day 2+	27 users	31% of loggers
↓		
6. Retained Day 7+	1 user	1% (early data)
↓		
7. Founding member (committed)	4 members	—

How Each Stage Works

Stage 1-2: Instagram → DM automation

- Founder posts trial reels/stories showing the bot in action
- CTA: "DM me 'hi' to get the link"
- Meta Business Suite "Custom Keywords" automation fires on "hi"
- Auto-reply contains: waitlist link (nutridesi.co) + beta sandbox link for those who can't wait

Stage 2-3: DM → Join

- The DM message gives two paths:
- **Primary:** Join the waitlist on nutridesi.co (captures name + WhatsApp number)
- **Secondary:** Try the beta immediately (sandbox join code)
- Waitlist signups are auto-inserted into the founding_members database
- Founder gets an instant WhatsApp alert on every signup

Stage 3-4: Join → First log (92% conversion)

- Once in the sandbox, the user just texts what they ate
- No onboarding, no setup, no profile creation required
- The 92% conversion from join → first log proves the zero-friction hypothesis

Stage 4-5: First log → Retention (31% D2+)

- 27 of 87 users came back a second day
- The bot logs food instantly, shows running totals, handles corrections
- Users who set a calorie/protein goal get a progress bar in every reply

Stage 6: D7 retention

- Target: 40% (industry benchmark for habit-forming products)
- Current: too early to measure reliably (most users joined in the last 2 weeks)
- The 1 user with 7+ days is a strong signal – they chose this over MyFitnessPal

The New Funnel (Waitlist-First, July 2026)

The original funnel sent everyone directly to the sandbox. Problem: no way to contact users when the permanent number launches, and no filtering of "just curious" vs "actually wants to track."

New funnel:



1. Reel/Story → DM automation → **waitlist first** (nutridesi.co)
2. Waitlist captures name + contact → auto-inserted into founding_members
3. Success screen reveals beta link → try immediately if they want
4. When WABA launches → founding members are migrated automatically

This gives us: a contact list, a committed cohort, and urgency (only 50 founding spots).

Key Numbers for the Pitch

Metric	Value	What it means
Total messages processed	1,314	Real usage, not test data
Unique users	94	Organic, zero ad spend
Join → first log	92%	Zero-friction hypothesis validated
D2+ retention	31%	Promising for an unpolished beta

Metric	Value	What it means
Food logs	775	People are actually tracking meals
Founding members	4	Committed users (free-for-life promise)
AI accuracy	100% (65/65)	Hinglish parsing works
Avg response time	1.7 seconds	Faster than opening any app

The Full System, Piece by Piece

1. WhatsApp Interface – Twilio Sandbox (currently) / WhatsApp Business API (migrating)

What it does: Receives messages from users' WhatsApp and sends replies back. **Why it's needed:** WhatsApp doesn't let random software read your messages. You need an official gateway – a company authorized by Meta to bridge WhatsApp messages to a server. Twilio is that company.

Current state: We use Twilio's "Sandbox" – a shared testing number (+1 415 523 8886) that's free but requires users to send a join code first. This is fine for beta testing but has limitations:

- Users must re-join every 72 hours of inactivity
- The number is shared with other Twilio developers (though messages are private)
- Cannot send proactive messages (daily summaries) outside a 24-hour window

Future state: A dedicated WhatsApp Business number (WABA) registered to NutriDesi. Users save it like a contact. No join codes, no expiry, can send daily summaries. Meta verification is in progress. **Cost:**

- Current (Sandbox): Free
 - Future (WABA): Service messages (user texts first, we reply) = free for first 1,000 conversations/month, then minimal. Daily summaries would cost ~₹0.12/message.
-

2. ngrok Tunnel – The Bridge Between the Internet and Our Computer

What it does: Creates a permanent public internet address (URL) that points to the computer sitting in our home/office. **Why it's needed:** When a user sends a WhatsApp message, Twilio needs to forward it to OUR server. But our server runs on a personal computer behind a home internet connection – it doesn't have a public address the way google.com does. ngrok solves this by creating a tunnel: it gives us a fixed URL (like abc123.ngrok.io) that routes internet traffic to our local machine. **Why not just use cloud hosting?** We will, eventually. For beta with 4–50 users, the local setup is:

- Faster to iterate (change code, restart in 6 seconds, live)
- Zero monthly hosting cost
- Good enough reliability for a beta test

Cost: Free (ngrok's free tier now includes one stable domain that survives restarts) **Risk:** If the home internet goes down, the bot goes down. Acceptable for beta; will move to cloud hosting before scaling past 50 users.

3. Node.js + Express – The Server (The Brain)

What it does: This is the actual software that processes messages. When Twilio forwards a user's WhatsApp message, this server:

1. Checks for spam/abuse (rate limits)
2. Prevents duplicate processing (Twilio sometimes sends the same message twice)
3. Sends the message to an AI model for understanding
4. Looks up calorie data
5. Calculates daily totals
6. Formats a reply
7. Sends the reply back through Twilio to the user's WhatsApp

What is Node.js? A runtime environment that lets us write server software in JavaScript (the most common programming language in the world). It's lightweight, fast, and handles many simultaneous users well – WhatsApp bots, chat systems, and real-time apps commonly use it. **What is Express?** A framework (a pre-built toolkit) on top of Node.js that handles the plumbing of receiving and responding to internet requests. Think of it like: Node.js is the engine, Express is the steering wheel and dashboard. Without it, we'd write 10x more code to do the same thing. **Why these choices?**

- Industry standard for real-time messaging applications
- Massive ecosystem of pre-built libraries (Twilio's official library, database connectors, AI SDKs)
- Single language for the entire backend = one developer can maintain everything
- Fast enough: our entire response pipeline (receive → AI → database → reply) takes 1.7–3 seconds

Cost: Free (it's open-source software). Runs on the Mac Mini.

4. AI Language Model (LLM) – The Parser

What it does: Understands casual, messy, Hindi-English food descriptions and converts them into structured data. **Why it's needed:** Users don't type "Chapati, quantity: 2, accompaniment: Dal Tadka, quantity: 1 bowl." They type "2 roti dal ke saath" or "had some chole bhature at lunch" or "ek plate rice medium wali." A traditional database lookup can't handle this. The AI model:

- Understands Hinglish (mixed Hindi-English)
- Recognizes that "roti", "chapati", "phulka", and "fulka" are the same food
- Knows that "do" before a food means "2" (Hindi), not the English verb
- Splits compound messages: "roti with ghee" → two items (roti + ghee) with separate calories
- Detects intent: is the user logging food, correcting a mistake, or asking a question?

Current provider: Google Gemini 3.1 Flash Lite

- Fastest available (1.7 seconds average response)
- Cheapest (\$0.25 per million input tokens)
- 100% accuracy on our 65-case test suite covering Hinglish, corrections, edge cases

Fallback chain: If the primary AI fails (server error, quota limit), the system automatically tries the next provider. Currently Gemini → Groq (Meta's Llama model) → Claude (Anthropic). Users never see an error; the chain fires silently in the background.

Why not build our own model? Training a custom model for Hinglish food parsing would cost \$50,000–200,000+ and take months. Using a general-purpose AI via API costs ₹0.13 per message and achieves 100% on our test suite today. At this stage, buying intelligence is

cheaper than building it. **Cost:**

- Current (50 users): ~₹2,000–3,000/month
 - At 300 DAU: ~₹6,000/month (\$72)
-

5. Supabase (PostgreSQL Database) – Memory

What it does: Stores everything the bot needs to remember:

- **User profiles:** phone number, daily calorie goal, preferences
- **Food logs:** every item logged, with calories, protein, carbs, fat, timestamp
- **Food reference:** 1,014 Indian recipes with lab-verified nutritional data (from the Indian Nutrient Databank, a government open-data source)
- **Founding members:** the waitlist-to-founding-member pipeline

Why Supabase? It's a managed PostgreSQL database (the industry-standard database used by companies from startups to banks). "Managed" means we don't maintain the database server – Supabase handles backups, security patches, uptime. We just use it. **Why not a spreadsheet or simple file?** At even 10 users logging 5 meals/day, that's 1,500 rows/month. We need:

- Fast lookups ("what did this phone number eat today?")
- Concurrent access (multiple users texting simultaneously)
- Structured queries ("total calories for user X on date Y")
- A proper food reference table with fuzzy text matching (finding "aaloo gobi" when the user types "aloo gobhi")

PostgreSQL handles all of this; a spreadsheet would collapse within a week.

Cost:

- Current (free tier): ₹0 – handles up to 500MB, which is ~2 years of data at 50 users
 - At 300 DAU: likely still free tier, or Pro plan at \$25/month (₹2,100) if we hit storage limits
-

6. The Mac Mini – Physical Server

What it does: The actual computer running the bot software. It's a Mac Mini (Apple's smallest desktop computer) sitting in the founder's home, connected to the internet 24/7.

Why a home computer instead of cloud hosting?

- Zero recurring cost (already owned)
- Faster development iteration (change code → restart in 6 seconds → live)
- Full control over the environment
- For 4–50 users, reliability requirements are modest

How it stays running:

- **launchd** (macOS's built-in process supervisor) monitors the bot. If it crashes, it's restarted automatically within 6 seconds. If the computer restarts (power cut, update), the bot starts automatically on login.
- **caffeinate** prevents the computer from sleeping (computers normally sleep when idle to save power – we need it awake 24/7)
- **Healthcheck script** pings the bot every 5 minutes. If it's down, the founder gets a WhatsApp alert immediately.
- **Power settings:** auto-restart after power failure is enabled at hardware level

Reliability track record: 99%+ uptime over 2 weeks of beta. The one outage (12 minutes) was caused by a code deployment, not infrastructure failure. **When do we outgrow this?** At 50+

active users, or when we need:

- Geographic redundancy (serving users closer to their region)
- Zero-downtime deployments
- SLA guarantees for investors/partners

The migration path is clear: deploy to Railway or Google Cloud Run (both support Node.js natively). Takes about 2 hours of work. Estimated cost: \$5-20/month.

7. Landing Site – nutridesi.co (Netlify)

What it does: The public website where potential users:

1. Learn what NutriDesi is
2. Join the waitlist (name + WhatsApp number)
3. Get access to the beta sandbox

Built with: Plain HTML/CSS (no framework). Hosted on Netlify (free tier), form submissions handled by Netlify Forms. **Waitlist automation:** When someone submits the form, the system automatically:

- Validates their contact (real phone number? email? Instagram handle?)
- Adds them to the founding_members table in the database
- Sends an instant WhatsApp alert to the founder

Cost: Free (Netlify free tier handles up to 100 form submissions/month, 100GB bandwidth)

8. Food Intelligence – The 4-Tier Lookup

Not every food is handled the same way. The system uses four tiers, from most accurate to least:

Tier	Source	Coverage	Accuracy
1. Curated list	Hand-verified by us	179 Indian staples + aliases	Highest (exact kcal/protein per serving)
2. Reference DB	Indian Nutrient Databank (govt)	1,014 recipes	High (lab-derived, per-100g)
3. AI estimate	The language model's knowledge	Unlimited	Medium ($\pm 20\%$, clamped to sane range)
4. Placeholder	Fixed 300 kcal	Everything else	Low but never zero

Key principle: The bot ALWAYS logs something. It never asks the user "how many calories do you think that was?" – that's the exact friction that kills habit formation. A $\pm 20\%$ estimate logged consistently is worth more than perfect data logged once a week.

9. Eval Suite – Quality Assurance

What it does: 65 test cases that cover every edge case we've encountered: Hinglish quantities, corrections, undo, brand names, modifiers, compound dishes. Before any

change to the AI prompt or model, we run the full suite. If accuracy drops below 100%, we don't ship. **Why it matters:** The AI model is a black box – you can't "see" if a change broke something by reading the code. You have to test it against known inputs and verify the outputs. This is our safety net. **Current score:** 65/65 (100%) on Gemini 3.1 Flash Lite

Cost Summary

Current State (4 founding members, beta)

Item	Monthly Cost	Notes
Gemini AI API	~₹300	Pay-per-use, minimal traffic
Twilio Sandbox	₹0	Free testing number
Supabase	₹0	Free tier
ngrok	₹0	Free tier (stable domain)
Netlify (website)	₹0	Free tier
Mac Mini electricity	~₹500	24/7 operation
Domain (nutridesi.co)	~₹100	Annual cost amortized
Total	~₹900/month	

Projected at 50 Users (founding members full)

Item	Monthly Cost	Notes
Gemini AI API	~₹2,500	50 users × 5 msgs/day
Twilio Sandbox	₹0	Still free
Supabase	₹0	Still within free tier
ngrok	₹0	Free tier
Mac Mini electricity	~₹500	Same
Total	~₹3,000/month	

Projected at 300 DAU (Growth Phase)

Item	Monthly Cost	Notes
Gemini AI API	~₹6,000	300 users × 5 msgs/day × 30 days = 45,000 calls
WhatsApp Business API	~₹1,000-3,000	Service messages mostly free; daily summaries ₹0.12/msg
Supabase Pro	~₹2,100	May need Pro plan for storage/bandwidth
Cloud hosting (Railway)	~₹500-1,700	Replaces Mac Mini for reliability

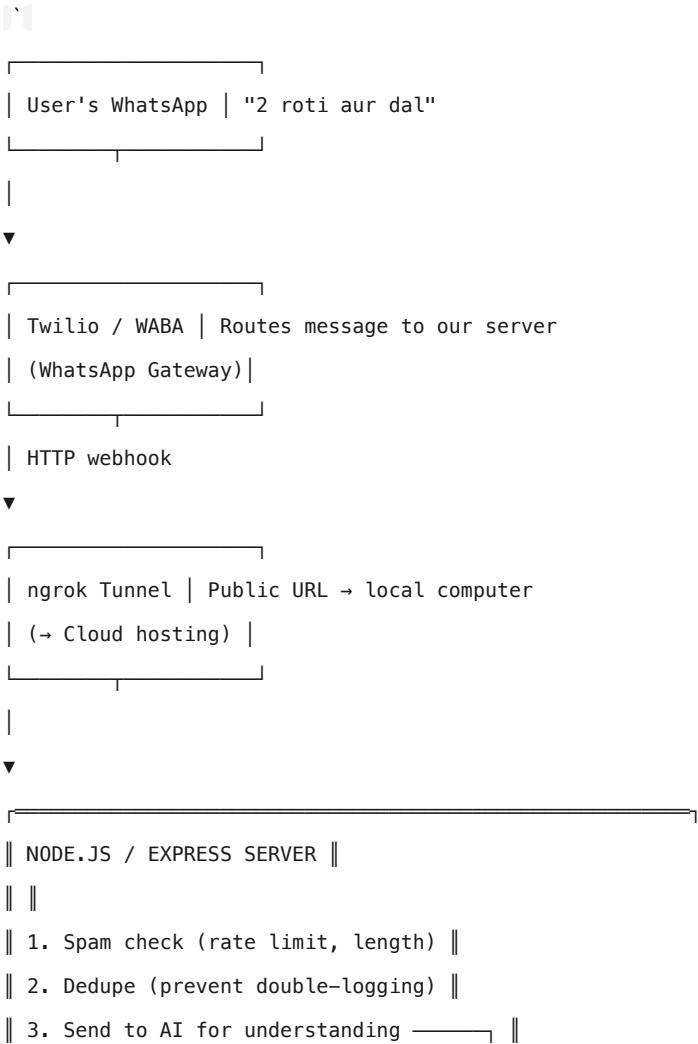
Item	Monthly Cost	Notes
Domain + SSL	~₹100	Same
Total	~₹10,000-13,000/month	(~\$120-155 USD)

Key Cost Insight

At 300 DAU, the total infrastructure cost is ₹10,000-13,000/month (~\$120-155). This is unusually low for a consumer AI product because:

- 1. User-initiated WhatsApp conversations are mostly free (Meta subsidizes business messaging)
 - 2. Gemini 3.1 Flash Lite is the cheapest available AI model at \$0.25/MTok
 - 3. No mobile app to maintain (no iOS/Android developer costs, no App Store fees)
 - 4. No user authentication system (phone number IS the identity)
- For comparison: a traditional calorie tracking app (iOS + Android + backend + AI) would cost ₹3-5 lakh/month to operate at 300 DAU.

Architecture Diagram (Simplified)




```
|| 4. Look up calories from database —| || |
|| 5. Calculate daily totals | ||
|| 6. Format reply | ||
|| ▼ ||
|| [ ||
|| | Gemini AI API | ||
|| | (understands | ||
|| | Hinglish food) | ||
|| [ ||
|| | ||
|| [ ||
|| | Supabase DB | ||
|| | (stores logs, | ||
|| | food data, | ||
|| | user profiles) | ||
|| [ ||
|| _____ ||
|
|
▼ (Twilio reply)
| [
| Twilio / WABA | Sends reply back
| ]
|
▼
| [
| User's WhatsApp | "✅ Logged: Roti x2 (178), Dal (180)
| | Today: 358 kcal · P18g C52g F8g"
| ]
| `
```

Risks and Mitigations

Risk	Likelihood	Impact	Mitigation
Mac Mini goes offline	Medium	Bot down until fixed	Healthcheck alerts + migration to cloud hosting planned
AI model deprecated/removed	Medium	Must switch models	Eval suite validates any new model in 2 minutes; already survived 2 model changes
Twilio sandbox limits hit	High (already happening)	Users can't message	WABA migration in progress (Meta verification submitted)

Risk	Likelihood	Impact	Mitigation
AI accuracy degrades	Low	Wrong calorie data	65-case eval suite catches regressions before deployment
Database exceeds free tier	Low	Need to upgrade	Pro plan is \$25/month – trivial cost
Competitor launches similar product	Medium	Market share	First-mover in Hinglish WhatsApp space; building retention before features

What's NOT Built (Deliberate Choices)

Feature	Why not yet
Mobile app	WhatsApp IS the app. Building iOS + Android = ₹5–10 lakh and 3–6 months. Validating retention first.
Photo-based food recognition	Camera AI for Indian food is unsolved at consumer quality. Text works now.
Web dashboard	Users don't need another screen to check. "What did I eat today?" query works in WhatsApp.
Personalized meal recommendations	Requires nutrition science expertise and liability considerations. Phase 2 if retention holds.
Hindi (Devanagari) input	Hinglish (Roman script) covers 90%+ of our target demographic's typing habits.

The Bottom Line for Decision-Makers

1. **Unit economics work:** ₹0.13/message AI cost. At ₹199/month subscription (planned), we need just 66 paying users to cover 300 DAU infrastructure costs.

1. **Defensibility:** The curated food database (179 items with Hinglish aliases), the correction framework (tested against 35 real incidents), and the eval suite (65 golden cases) are compounding assets. A competitor starting today would face the same 2 weeks of "bhelpuri = 530 kcal" bugs we already fixed.

1. **Migration path is clear:** Mac Mini → Cloud (2 hours), Twilio Sandbox → WABA (in progress), Single developer → team (codebase is documented, tested, and version-controlled).

1. **The metric that matters:** D7 retention $\geq$ 40% validates the hypothesis. We're tracking this with the current founding members.

Document prepared July 2026. For technical architecture details, see [docs/architecture.html](#) in the codebase.